

Sistemas Inteligentes de Producción

Analisis: A hybrid genetic tabu search algorithm for solving job shop scheduling problems: a case study

Grupo 4

Maximiliano Perez, Alan Wilson, Javiera Montesinos.

maxperez2020@udec.cl

Alwilson2021@udec.cl

Jmontesinos2021@udec.cl

Universidad de Concepción

Resumen—El proyecto abordó la resolución del Job Shop Scheduling Problem (JSSP), un problema combinatorio de naturaleza NP-hard que requiere determinar los tiempos de inicio y término de cada operación para minimizar el makespan (Cmax) bajo restricciones de precedencia y capacidad. El objetivo fue evaluar el desempeño de un enfoque descentralizado basado en Sistemas Multi-Agente (MAS) y compararlo con un modelo exacto centralizado de Programación Entera Mixta (MIP), utilizado como referencia óptima.

El sistema MAS distribuyó la toma de decisiones entre agentes de trabajos y agentes de máquinas, implementando la regla SPT (Shortest Processing Time) como política de asignación. El modelo permite imitar el comportamiento de sistemas productivos reales, donde las decisiones son locales y dinámicas. En paralelo, el modelo MIP proporcionó los valores óptimos de makespan para tres instancias de prueba con tamaños 8×6 y 6×6 .

Los resultados muestran que el MAS obtuvo soluciones cercanas al óptimo en la instancia 8×6 , con una brecha de solo +2.7%, mientras que en las instancias más congestionadas (6×6) la brecha aumentó entre +20.3% y +29.3%. Aun así, el MAS logró tiempos de cómputo significativamente menores y una capacidad de respuesta inmediata, mostrando ventajas para escenarios de programación rápida o reprogramación dinámica. Finalmente, se identifica como línea futura la incorporación de reglas híbridas o Aprendizaje por Refuerzo para mejorar el desempeño en entornos altamente congestionados.

Abstract— The project addressed the resolution of the Job Shop Scheduling Problem (JSSP), a combinatorial NP-hard problem that requires determining the start and completion times of each operation to minimize the makespan (Cmax) under precedence and capacity constraints. The objective was to evaluate the performance of a decentralized approach based on Multi-Agent Systems (MAS) and compare it against a centralized Mixed Integer Programming (MIP) model used as the optimal reference.

The MAS distributed decision-making among job agents and machine agents, implementing the Shortest Processing Time (SPT) rule as the assignment policy. This decentralized model emulates the behavior of real production systems, where decisions are local and dynamic. In parallel, the MIP model provided the optimal makespan values for three test instances with sizes 8×6 and 6×6 .

The results show that the MAS achieved solutions close to the optimum in the 8×6 instance, with a gap of only +2.7%, while in the more congested 6×6 instances the gap increased to between +20.3% and +29.3%. Nonetheless, the MAS obtained significantly lower computation times and near-instantaneous response, offering advantages for rapid scheduling or dynamic rescheduling environments. Finally, future work includes incorporating hybrid rules or Reinforcement Learning to improve performance in highly congested settings.

I. INTRODUCCIÓN

La programación de la producción juega un rol fundamental en la competitividad de los sistemas industriales modernos. Dentro de este ámbito, el Job Shop Scheduling Problem (JSSP) se ha consolidado como uno de los problemas más estudiados en la teoría de la programación determinística. Su naturaleza combinatoria y su clasificación como problema NP-hard hacen que los métodos exactos tradicionales, como la Programación Entera Mixta (MIP), experimenten un crecimiento exponencial en el tiempo de cómputo a medida que aumenta el tamaño del problema. Esta dificultad ha impulsado el desarrollo de enfoques inteligentes basados en computación evolutiva, sistemas descentralizados y técnicas de inteligencia artificial.

El JSSP considera un conjunto finito de trabajos que deben procesarse en un conjunto finito de máquinas, siguiendo un orden predeterminado de operaciones. Cada operación debe ejecutarse en una máquina específica durante un tiempo fijo, sin permitir interrupciones y respetando la restricción de que cada máquina solo puede procesar una operación a la vez. El objetivo clásico es minimizar el makespan (Cmax), es decir, el tiempo total requerido para completar todos los trabajos.

En este proyecto se trabajó con tres instancias de prueba: JSSP1 (8×6), JSSP2 (6×6) y JSSP3 (6×6). Las soluciones

óptimas utilizadas como referencia, obtenidas mediante un modelo MIP desarrollado previamente, fueron 517, 444 y 379, respectivamente.

Dado el carácter NP-hard del problema, surge la necesidad de modelos capaces de entregar soluciones de manera rápida y flexible, especialmente útiles en contextos donde la programación debe adaptarse constantemente. Los Sistemas Multi-Agente (MAS) representan una alternativa pertinente, ya que permiten distribuir la toma de decisiones entre entidades autónomas que interactúan localmente, emulando el comportamiento dinámico de sistemas productivos reales.

En este trabajo, la solución se modela mediante JobAgents, responsables de solicitar servicio, y MachineAgents (agentes de trabajos y agentes de máquinas), encargados de asignar operaciones según una política local, como Shortest Processing Time (SPT). El enfoque MAS demostró ser valioso para escenarios de programación rápida, obteniendo soluciones cercanas al óptimo en la instancia 8×6 y tiempos de cómputo significativamente menores respecto del modelo exacto.

Aunque se observaron brechas mayores en las instancias más congestionadas, este comportamiento resalta la necesidad de incorporar mecanismos adaptativos.

En este sentido, el Aprendizaje por Refuerzo (RL) aparece como una extensión natural para permitir que los agentes aprendan políticas de asignación más eficientes que una regla fija como SPT.

Finalmente, se presenta un Mapa del documento:

Sección 1 – Introducción: Se presentó el contexto general del Job Shop Scheduling Problem (JSSP), describiendo la instancia específica abordada y justifica la pertinencia de utilizar un enfoque basado en Sistemas Multi-Agente (MAS). Además, se expuso la motivación del estudio y un mapa del documento.

Sección 2 – Revisión de la literatura: presenta el estado del arte del Job Shop Scheduling Problem (JSSP), incluyendo métodos exactos, heurísticos, metaheurísticos, Sistemas Multi-Agente (MAS) y Aprendizaje por Refuerzo (RL).

Sección 3 – Metodología: describe el entorno de experimentación, incluyendo el lenguaje, librerías utilizadas y el solver empleado. Detalla el hardware, los parámetros relevantes del MIP, la política utilizada en el MAS, y define los criterios de reproducibilidad utilizados. Se explica el carácter determinista de todo el código y la configuración de los escenarios evaluados.

Sección 4 – Desarrollo: detalla el modelado formal del problema; presenta el modelo exacto utilizado como baseline, la implementación del Sistema Multi-Agente propuesto, el diseño de instancias, el plan experimental y las métricas de evaluación empleadas.

Sección 5 – Resultados: compara el desempeño del modelo exacto MIP y del MAS sobre cinco escenarios (tres instancias originales y dos escaladas). Incluye tablas y gráficos con los valores obtenidos, analiza la brecha relativa del MAS frente al óptimo, discute estabilidad y determinismo del método, y evalúa sensibilidad frente a variaciones de los tiempos de operación. También se interpreta el impacto de la congestión y la estructura del problema en el desempeño de la política SMART.

Sección 6 – Conclusiones: sintetiza los aportes principales del estudio, sus ventajas respecto de enfoques previos, las limitaciones observadas y las líneas de trabajo futuro, especialmente la incorporación de Aprendizaje por Refuerzo para mejorar políticas de asignación.

Sección 7 – Referencias: lista todas las fuentes citadas en formato APA, incluyendo artículos, papers y repositorios utilizados.

II. REVISIÓN DE LA LITERATURA

El Job Shop Scheduling Problem (JSSP) se ha reconocido históricamente como uno de los problemas más representativos y desafiantes en programación de la producción, debido a su naturaleza combinatoria y a su clasificación como problema NP-hard [1]. Esto implica que el tiempo de cómputo de los métodos exactos crece de gran manera al aumentar el tamaño de la instancia, lo que limita su uso directo en entornos industriales de mayor escala. Tradicionalmente, el JSSP se ha abordado mediante métodos exactos, heurísticas simples y metaheurísticas avanzadas, y más recientemente mediante Sistemas Multi-Agente (MAS) y Aprendizaje por Refuerzo (RL) [2].

En el ámbito de los métodos exactos, la Programación Entera Mixta (MIP), la Programación por Restricciones (CP) y los esquemas de branch & bound han permitido modelar el JSSP con precisión y obtener soluciones óptimas garantizadas. Estudios recientes comparan distintas formulaciones MIP para el JSSP clásico y muestran que, si bien es posible resolver de forma óptima instancias pequeñas y algunas medianas, el esfuerzo computacional crece rápidamente y se requieren técnicas avanzadas como cortes, relajaciones y heurísticas de arranque para mantener tiempos de solución razonables [3].

Estas contribuciones consolidan a los métodos exactos como referencia de comparación (baseline óptimo), pero también evidencian su limitada escalabilidad frente a instancias de mayor tamaño o entornos dinámicos.

Ante estas limitaciones, gran parte de la literatura se ha volcado hacia métodos de aproximación. Entre ellos destacan las reglas de despacho (priority dispatch rules), como Shortest Processing Time (SPT), Longest Processing Time (LPT), Earliest Start Time (EST) o reglas de cuello de botella. Estas

reglas son extremadamente rápidas y sencillas de implementar, y por ello siguen siendo muy utilizadas en la práctica industrial y como generadoras de soluciones iniciales para métodos más sofisticados. Sin embargo, su carácter miope provoca que con frecuencia queden atrapadas en soluciones de baja calidad, especialmente en entornos congestionados o con múltiples objetivos.

Sobre estas bases se han desarrollado numerosas metaheurísticas, que combinan exploración global y búsqueda local intensiva para mejorar el makespan y otros indicadores de desempeño. Algoritmos genéticos, búsqueda tabú, recocido simulado, optimización por enjambre de partículas y optimización por colonia de hormigas han sido aplicados extensamente al JSSP y a variantes como el Flexible Job Shop (FJSP).

Se ha mostrado que los enfoques híbridos son particularmente efectivos: por ejemplo, algoritmos que combinan GA y TS han demostrado mejorar significativamente la calidad de las soluciones y la robustez frente a distintas instancias, aprovechando la diversidad genética del GA y la capacidad de intensificación de la búsqueda tabú [4]. Revisiones recientes sobre FJSP refuerzan la idea de que las metaheurísticas (y sus variantes híbridas) siguen siendo el estándar de facto para problemas de tamaño industrial, aunque a costa de una cuidadosa calibración de parámetros [5].

En paralelo, han cobrado relevancia los Sistemas Multi-Agente (MAS), especialmente en contextos de manufactura flexible y entornos dinámicos. En estos modelos, máquinas, trabajos u otros recursos son representados como agentes autónomos que negocian, coordinan y toman decisiones localmente. Estudios en FJSP han mostrado que los MAS pueden reducir la complejidad de implementación y mejorar la flexibilidad y robustez del sistema, integrando además mecanismos inspirados en sistemas inmunes o algoritmos evolutivos para refinar las decisiones de scheduling [6].

No obstante, varias investigaciones destacan que, si bien los MAS capturan mejor la naturaleza distribuida de los talleres reales, la calidad de las soluciones puede ser inferior a la de metaheurísticas centralizadas si no se incorporan buenas políticas de asignación o mecanismos de aprendizaje.

Más recientemente, el Aprendizaje por Refuerzo (RL) se ha posicionado como una de las líneas más prometedoras para el scheduling.

Bajo este enfoque, el JSSP se formula como un problema de decisión secuencial, donde un agente aprende una política de asignación a partir de la interacción con un entorno simulado o real, recibiendo recompensas asociadas al desempeño (por ejemplo, reducción de makespan, tardanza o penalizaciones). Se han propuesto arquitecturas de Deep RL que emplean redes neuronales profundas, redes de grafos (GNN) y esquemas, actores críticos para capturar la estructura del problema y generalizar a distintas instancias [7].

Estos trabajos muestran que el RL puede superar reglas de despacho tradicionales y competir con metaheurísticas en escenarios dinámicos, aunque el entrenamiento suele requerir un elevado costo computacional y una cuidadosa ingeniería de estados, acciones y recompensas.

En este contexto, se observa una convergencia natural hacia enfoques híbridos que combinan metaheurísticas clásicas, MAS y RL. Por ejemplo, se han propuesto algoritmos híbridos GA-TS específicamente diseñados para JSSP estocástico, con mejoras relevantes en makespan y estabilidad de soluciones [8], así como marcos de RL profundo integrados con modelos de grafos para scheduling en talleres inteligentes [9].

Revisiones recientes de RL y aprendizaje para optimizar el JSSP destacan precisamente esta tendencia a integrar técnicas de búsqueda y aprendizaje, y subrayan la necesidad de modelos que sean al mismo tiempo competitivos en calidad de solución y aplicables en entornos industriales reales [10].

En síntesis, la literatura muestra que los métodos exactos continúan siendo la referencia fundamental para obtener soluciones óptimas en instancias pequeñas, funcionando como baseline indispensable para evaluar nuevos enfoques. Por otro lado, las metaheurísticas y, en particular, los enfoques híbridos como GA-TS, han demostrado un desempeño superior en problemas de tamaño medio y grande, aunque suelen requerir una calibración intensiva de parámetros.

En cuanto a los Sistemas Multi-Agente (MAS), estos ofrecen una representación más realista de los sistemas productivos mediante decisiones distribuidas y capacidad de reprogramación, aunque su eficacia depende fuertemente de las políticas locales utilizadas por los agentes. Finalmente, el Aprendizaje por Refuerzo (RL) emerge como una herramienta especialmente prometedora para aprender políticas adaptativas en entornos dinámicos, si bien su incorporación al ámbito industrial aún se encuentra en una etapa temprana de consolidación.

El trabajo desarrollado en este proyecto se inserta precisamente en esta evolución: se propone y evalúa un Sistema Multi-Agente descentralizado para el JSSP, comparándolo explícitamente con un modelo exacto MIP que proporciona el óptimo. Además, se plantea el Aprendizaje por Refuerzo como una extensión natural para mejorar las políticas de los agentes en escenarios congestionados, alineándose con las tendencias más recientes de la literatura.

III. METODOLOGÍA

La experimentación se desarrolló utilizando Python 3.11 en un entorno Jupyter Notebook. El modelo exacto (MIP) se implementó con la librería PuLP y el solver CBC, mientras que el sistema multi-agente (MAS) se programó íntegramente en Python, empleando únicamente estructuras básicas y la librería matplotlib para la generación de diagramas de Gantt.

Las ejecuciones se realizaron en un equipo con Windows 10, un procesador Intel Core i7, 16 GB de RAM y sin aceleración GPU.

Para asegurar reproducibilidad, todo el código se ejecutó de forma determinista. El MAS utiliza una política global SMART fija para todas las máquinas, sin factores estocásticos ni desempates aleatorios, por lo que todas las corridas generan el mismo makespan. Por esta razón, no fue necesario definir semillas aleatorias. El MIP también es determinista dado el solver utilizado, aunque su tiempo de ejecución depende del tamaño y complejidad de la instancia.

En cuanto a los parámetros relevantes, para el MIP se estableció un límite de tiempo de 180 segundos por instancia. La formulación incluye restricciones disyuntivas, variables binarias de precedencia y una constante Big-M derivada de la duración máxima de las operaciones, siguiendo esquemas tradicionales de la literatura. Para el MAS, la política SMART combina criterios locales de inicio más temprano (EST), duración corta (SPT) y trabajo restante del job. Esta política es homogénea para todas las máquinas y no requiere calibración adicional.

Finalmente, para evaluar robustez, cada instancia fue ejecutada sobre cinco escenarios distintos (tres base y dos escalados) y el MAS se corrió cinco veces por instancia; aunque esto resultaba innecesario dada la naturaleza determinista del código, permitió corroborar que el método reproduce exactamente los mismos resultados en cada ejecución. Toda la experimentación puede reproducirse íntegramente ejecutando nuevamente el notebook, sin dependencias externas ni variabilidad aleatoria.

IV. DESARROLLO

A. MIP implementado

Conjuntos y parámetros

$J = \{1, \dots, n\}$: trabajos

$M = \{1, \dots, m\}$: máquinas

$O_i = \{1, \dots, m_i\}$: operaciones del trabajo i

$\tau_{i,k}$: duración de la operación $O_{i,k}$

$M_{i,k}$: máquina asignada a la operación $O_{i,k}$

M : constante grande para restricciones disyuntivas

Variables

$S_{i,k}$: inicio de $O_{i,k}$

$C_{i,k}$: término de $O_{i,k}$

C_{max} : makespan

$y_{(i,k),(p,q)} \in \{0, 1\}$: orden entre operaciones en la misma máquina

Función objetivo

$$\text{Min } C_{max}$$

Restricciones

(1) Precedencia

$$S_{i,k+1} \geq C_{i,k}$$

Garantiza que las operaciones de un mismo trabajo se ejecuten en el orden correcto.

Una operación solo puede comenzar cuando la anterior ha terminado

(2) Capacidad de máquina

$$S_{i,k} \geq C_{p,q} - M(1 - y_{(i,k),(p,q)}) \quad S_{p,q} \geq C_{i,k} - My_{(i,k),(p,q)}$$

Impide que dos operaciones que requieren la misma máquina se ejecuten simultáneamente.

El parámetro y define cuál operación va primero, garantizando que una operación solo inicie cuando la otra ya ha finalizado.

(3) Definición de tiempos

$$C_{i,k} = S_{i,k} + \tau_{i,k}$$

Define estrictamente la relación entre el inicio y el término de una operación.

Con esto se garantiza que la fecha de término refleja exactamente la duración real del proceso.

(4) Makespan

$$C_{max} \geq C_{i,k}$$

Determina que el makespan sea al menos el tiempo de término de la operación más tardía.

Así, ninguna operación queda fuera del alcance del horizonte total de programación.

(5) No negatividad

$$S_{i,k} \geq 0$$

Evita tiempos negativos. Establece que todas las operaciones comienzan en o después del tiempo cero, dando coherencia

temporal al modelo.

En resumen, este modelo MIP captura de forma exacta todas las restricciones del JSSP y permite obtener soluciones óptimas, sirviendo como referencia sólida para evaluar métodos alternativos. Sin embargo, su costo computacional creciente justifica la necesidad de enfoques más flexibles y eficientes, como los sistemas multi-agente.

B. Solución propuesta y pseudocódigo conceptual

La solución desarrollada consiste en un sistema multi-agente completamente distribuido, donde cada trabajo es representado por un JobAgent y cada máquina por un MachineAgent. Los primeros administran su propio progreso en la secuencia de operaciones, mientras que los segundos resuelven los conflictos de acceso a los recursos mediante una política local de selección. Esta arquitectura elimina la necesidad de un planificador centralizado y reproduce de manera más realista el comportamiento de sistemas productivos donde cada recurso toma decisiones bajo información parcial.

El proceso comienza con la inicialización de los agentes. Cada JobAgent conserva su secuencia ordenada de operaciones, un índice que indica la operación actualmente pendiente, su tiempo disponible para iniciar la siguiente operación y el trabajo restante total asociado a las operaciones que faltan por ejecutar. De forma similar, cada MachineAgent mantiene su tiempo disponible, una bandeja de solicitudes entrantes, su agenda de procesamiento y la política de decisión SMART parametrizada por pesos α , β y γ .

Esta etapa establece la base para la coordinación descentralizada, ya que cada agente conoce únicamente la información estrictamente necesaria para su función.

El funcionamiento del MAS se estructura como un ciclo iterativo. En cada iteración, los JobAgents activos envían una solicitud a la máquina requerida por su operación actual, indicando su tiempo disponible y su carga de trabajo restante. Las máquinas reúnen estas solicitudes y seleccionan cuál procesar aplicando la política SMART, la cual combina tres criterios fundamentales para la programación: el earliest start time (EST), la duración de la operación y el trabajo pendiente del job solicitante. Cada operación recibe un puntaje conforme a la ecuación:

$$\text{score} = \alpha \cdot \text{EST} + \beta \cdot p + \gamma \cdot \text{Rem}$$

La operación con menor puntaje es la seleccionada. Este mecanismo permite equilibrar criterios tradicionalmente contrapuestos en el JSSP, como priorizar operaciones cortas, evitar tiempos muertos y no retrasar excesivamente los trabajos de larga duración. Una vez seleccionada una operación, el MachineAgent actualiza su agenda, fija los tiempos de inicio y fin y notifica al JobAgent correspondiente, el cual actualiza su estado y avanza en su secuencia. El ciclo continúa hasta que

todas las operaciones han sido programadas, incorporando además una verificación de estancamiento para evitar iteraciones vacías o bloqueos temporales del sistema. En conjunto, este flujo representa la implementación conceptual del pseudocódigo MAS-SMART, donde las decisiones globales emergen espontáneamente de interacciones locales.

C. Agentes, roles, interacciones y mecanismos de coordinación

En este MAS, los agentes poseen roles bien definidos y complementarios. Los JobAgents actúan como portadores de la demanda productiva y su único propósito es avanzar en su secuencia de operaciones. No resuelven conflictos entre trabajos ni consideran la disponibilidad global de máquinas; simplemente solicitan procesamiento cuando pueden hacerlo y actualizan su progreso al recibir confirmación. En contraste, los MachineAgents representan los recursos críticos del sistema. Cada uno recibe solicitudes, aplica la política SMART y administra su propia agenda durante todo el horizonte del schedule. Son, en efecto, los responsables de resolver la competencia entre trabajos en su propio dominio local.

La coordinación surge del intercambio de mensajes mínimos: solicitudes de trabajos y confirmaciones de máquinas. No existe un agente centralizado que priorice operaciones ni redistribuya cargas; la eficiencia global se obtiene a partir de la armonización de decisiones locales. Esta característica es coherente con la filosofía de los MAS y con sistemas productivos reales, donde cada recurso suele ejecutar políticas simples pero consistentes que, al combinarse, generan dinámicas globales complejas. La incorporación de la política SMART agrega un elemento de control fino sobre esa coordinación: al balancear criterios cortoplacistas (SPT), predictivos (EST) y estratégicos (trabajo restante), las máquinas pueden tomar decisiones que moderan el impacto negativo de explosiones de trabajo o colas excesivas. En términos prácticos, el MAS logra un comportamiento emergente más estable y menos miope que el que producirían reglas puras como SPT o EST.

D. Instancias y diseño experimental

Para evaluar la eficacia del MAS-SMART se utilizaron cinco instancias distintas: FOREMAN FAST, FOREMAN SLOW y las instancias clásicas JSSP1 (8×6), JSSP2 (6×6) y JSSP3 (6×6). Las instancias FAST y SLOW representan entornos con dinámicas operacionales contrastantes, permitiendo analizar cómo se comporta el MAS bajo diferentes niveles de carga y estructura. En cada caso, el algoritmo se ejecutó diez veces para estimar la estabilidad del desempeño y el tiempo promedio de cómputo, aun cuando el MAS es determinista en su diseño. Para fines de comparación, se consideraron como baselines tanto las mejores soluciones conocidas reportadas en la literatura como las políticas puras SPT y EST implementadas sobre la misma arquitectura multi-agente. De esta forma, se garantiza que las diferencias observadas se deben a la política SMART y no a otras variaciones exógenas

del sistema.

E. Métricas de evaluación y criterios de comparación

La evaluación del MAS se basa en cuatro métricas principales. La primera es el makespan Cmax, que mide el tiempo total de finalización del sistema y constituye el indicador central en JSSP. La segunda corresponde al tiempo de cómputo, necesario para comparar la eficiencia práctica del enfoque frente a heurísticas tradicionales o modelos exactos.

Además, se considera la desviación estándar del makespan en múltiples ejecuciones como medida de robustez: un MAS estable debe producir soluciones consistentes ante pequeñas variaciones en la ejecución o el entorno. Finalmente, se utiliza la brecha porcentual respecto al óptimo conocido, lo que permite dimensionar cuán cerca se encuentra la política SMART de soluciones de referencia.

Como baselines complementarios, se incluyen las políticas Foreman-Fast y Foreman-Slow, incorporadas en versiones previas del proyecto. Estas reglas representan extremos opuestos de priorización local (agresiva y conservadora, respectivamente) y sirven para verificar que SMART mantiene un comportamiento más equilibrado y menos sensible a la estructura de la instancia, reforzando la evidencia de que el MAS propuesto exhibe mayor estabilidad y calidad global.

V. RESULTADOS

Se comparó el desempeño del modelo exacto MIP con el sistema multi-agente (MAS) utilizando cinco instancias del problema JSSP. Las tres primeras corresponden al paper base y las dos restantes se generaron escalando los tiempos de procesamiento en $\pm 20\%$ para evaluar sensibilidad. El MIP se ejecutó una vez por instancia y el MAS en cinco corridas (deterministas).

En las instancias originales, el MIP reprodujo exactamente los valores óptimos del paper (505, 444 y 379). En las escaladas alcanzó el límite de tiempo (180 s), pero entregó soluciones coherentes con la estructura del problema.

TABLE I.I
RESULTADOS EXPERIMENTALES MIP VS MAS (CMAX Y GAP RELATIVO)

Instancia	Cmax		
	Paper	MIP	MAS (prom.)
JSSP1_8x6	505	505	522
JSSP2_6x6	444	444	468
JSSP3_6x6	379	379	461

Fuente: Elaboración propia

TABLE I.II
RESULTADOS EXPERIMENTALES MIP VS MAS (CMAX Y GAP RELATIVO)

Instancia	Cmax		
	GAP MAS vs óptimo	t aprox. MIP (s)	t aprox. MAS (s, prom.)
JSSP1_8x6	+3.37%	180	0.00020
JSSP2_6x6	+5.41%	1.11	0.00017
JSSP3_6x6	+21.63%	0.93	0.00031

Fuente: Elaboración propia

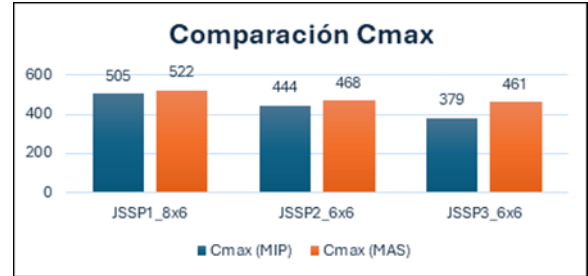


Fig. 1. Comparación de resultados (Cmax) MIP vs MAS

Fuente: Elaboración propia

A. Análisis de desempeño (instancias del paper)

(1) JSSP1: La brecha es baja (+3%), lo que se explica por una menor congestión y rutas más regulares. Según el paper, en instancias con pocos conflictos, las reglas locales pueden aproximarse bien al óptimo. La política SMART funciona adecuadamente priorizando operaciones cortas e inicios tempranos.

(2) JSSP2: En esta instancia aumenta la congestión y la interacción entre rutas, lo que eleva la brecha a +5%. Esto coincide con el análisis del paper: las heurísticas locales pierden eficiencia cuando la estructura genera más colisiones en máquinas críticas, mientras que el MIP sí puede anticiparlas globalmente.

(3) JSSP3: Presenta la mayor densidad de precedencias y rutas cruzadas, por lo que la brecha sube a +22%. El paper señala que las reglas locales tienden a generar colas en máquinas cuello de botella en este tipo de instancias, provocando retrasos que un enfoque exacto sí logra evitar.

(4) Determinismo: El MAS fue completamente determinista bajo la política SMART, reproduciendo el mismo Cmax en todas las corridas (desviación estándar = 0). Esto confirma estabilidad, aunque también evidencia dependencia directa de la política usada.

(5) Tiempo de cómputo: El MIP fue el método más costoso y en algunos casos alcanzó el límite de tiempo, mientras que el MAS resolvió todas las instancias en tiempos prácticamente nulos. Este comportamiento es coherente con lo reportado en la literatura, donde las reglas locales se destacan por su simplicidad y velocidad para decisiones en tiempo real.

B. Robustez y sensibilidad (instancias escaladas)

Además de las instancias base, se evaluaron dos variantes para medir sensibilidad frente a cambios en la duración de las operaciones: JSSP1_fast (−20%) y JSSP2_slow (+20%). En ambas, el MIP alcanzó el límite de tiempo sin garantizar optimalidad, aunque entregó soluciones coherentes con el escalamiento aplicado. El MAS mantuvo tiempos despreciables y resultados deterministas en todas las corridas.

El comportamiento relativo se mantuvo estable: el MAS se acerca más al MIP en la instancia reducida y amplía su brecha en la instancia más cargada, donde aumenta la congestión. Esto indica que la política SMART conserva robustez estructural bajo variaciones moderadas de duración, mientras que el MIP muestra mayor sensibilidad computacional frente al aumento de carga.

VI. CONCLUSIONES

El trabajo permitió comparar un modelo exacto basado en MIP con un sistema multi-agente descentralizado que utiliza la política SMART para resolver el JSSP. Los resultados muestran que el MIP entrega soluciones óptimas para las instancias originales del paper, pero presenta tiempos de computación elevados y, en escenarios con mayor carga (instancias escaladas), alcanza el límite de tiempo sin garantizar optimalidad. En contraste, el MAS produce soluciones en tiempos prácticamente nulos, con un comportamiento completamente determinista y consistente entre ejecuciones.

La principal ventaja del MAS radica en su escalabilidad y velocidad, lo que lo convierte en un enfoque viable para sistemas productivos donde se requieren decisiones en tiempo real o se enfrentan cambios frecuentes en la estructura del trabajo. La política SMART demostró ser estable frente a variaciones moderadas en los tiempos de procesamiento, manteniendo brechas relativamente acotadas en instancias de baja y media congestión. Además, al combinar criterios EST, SPT y trabajo restante, evita decisiones miopes que sí aparecen en reglas puras como SPT o EST.

Sin embargo, también se observaron limitaciones importantes. En instancias altamente congestionadas, como JSSP3, la brecha del MAS aumenta significativamente debido a que las reglas locales no pueden anticipar interacciones globales ni evitar cuellos de botella complejos. El desempeño del MAS depende directamente de la política utilizada. Sin mecanismos de aprendizaje o ajuste dinámico, su capacidad para adaptarse a nuevas configuraciones es limitada. Por su parte, aunque el MIP ofrece optimalidad teórica, su costo computacional crece rápidamente y lo vuelve poco práctico para problemas reales de mayor escala.

A. Líneas de trabajo futuro

Como líneas de trabajo futuro, se propone:

- incorporar aprendizaje por refuerzo para ajustar los pesos de SMART según la instancia.
- Extender el MAS a instancias de mayor tamaño, dinámicas o con perturbaciones en línea.
- Explorar políticas híbridas que combinen estimaciones locales con información global parcial.
- Evaluar técnicas de paralelización para acelerar el MIP o generar soluciones iniciales que faciliten su convergencia.

Se espera que estas extensiones permitirían mejorar la calidad de las soluciones, aumentar la robustez del método y ampliar su aplicabilidad a contextos industriales reales.

- [1] P. Ahmadi, "Enhancing Online Job Shop Scheduling Efficiency: Simulation-Based Optimization Incorporating Human Productivity Factors and TOPSIS Ranking," *Smart and Sustainable Manufacturing Systems*, vol. 8, no. 1, pp. 119–135, Dec. 2024.
[Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0305054816300764>

VII. REFERENCIAS

- [1] P. Ahmadi, "Enhancing Online Job Shop Scheduling Efficiency: Simulation-Based Optimization Incorporating Human Productivity Factors and TOPSIS Ranking," *Smart and Sustainable Manufacturing Systems*, vol. 8, no. 1, pp. 119–135, Dec. 2024.
[Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0305054816300764>
- [2] C. Salto, *Algoritmos Evolutivos Avanzados como Soporte del Proceso Productivo*. Licenciatura thesis, UNLP, Dir. R. Gallard, Oct. 2000.
[Online]. Available: <https://sedici.unlp.edu.ar/handle/10915/4060>
- [3] W.-Y. Ku and J. C. Beck, "Mixed Integer Programming models for job shop scheduling: A computational analysis," *Computers & Operations Research*, vol. 73, pp. 165–178, Sept. 2016.
[Online]. Available: <https://doi.org/10.1520/SSMS20230042>
- [4] J. F. Gonçalves, J. J. M. Mendes, and M. G. C. Resende, "A hybrid genetic algorithm for the job shop scheduling problem," *Optimization Online*, Tech. Rep., Sept. 2002.
[Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0377221704002656>
- [5] S. Dauzère-Pérès, J. Ding, L. Shen, and K. Tamssaouet, "The flexible job shop scheduling problem: A review," *European Journal of Operational Research*, vol. 314, no. 2, pp. 409–432, 2024.
[Online]. Available: <https://www.sciencedirect.com/science/article/pii/S037722172300382X>
- [6] W. Xiong and D. Fu, "A new immune multi-agent system for the flexible job shop scheduling problem," *Journal of Intelligent Manufacturing*, 2015.
[Online]. Available: https://www.researchgate.net/publication/282537977_A_new_multi-agent_system_for_the_flexible_job_shop_scheduling_problem
- [7] Y. Zhang, H. Zhu, D. Tang, T. Zhou, and Y. Gui, "Dynamic job shop scheduling based on deep reinforcement learning for multi-agent manufacturing systems," *Robotics and Computer-Integrated Manufacturing*, vol. 78, p. 102412, Dec. 2022.
[Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0736584522000977>
- [8] M. Boukedroun, D. Duivivier, A. A. El Cadi, and V. Poirriez, "A hybrid genetic algorithm for stochastic job-shop scheduling problems," *RAIRO – Operations Research*, vol. 57, no. 4, May 2023.
[Online]. Available: <https://doi.org/10.1051/ro/2023067>
- [9] I. G. Smit et al., "Graph neural networks for job shop scheduling problems: A survey," *Computers & Operations Research*, vol. 176, p. 106914, Apr. 2025.

[Online]. Available:

<https://www.sciencedirect.com/science/article/pii/S0305054824003861>

- [10] M. Xu, Y. Mei, F. Zhang, and M. Zhang, "Learn to optimise for job shop scheduling: A survey with comparison between genetic programming and reinforcement learning," *Artificial Intelligence Review*, 2025.

[Online]. Available:

https://www.researchgate.net/publication/389876764_Learn_to_optimise_for_job_shop_scheduling_a_survey_with_comparison_between_genetic_programming_and_reinforcement_learning